

Duyệt đồ thị và đường đi ngắn nhất

BFS, DFS và thuật toán Dijkstra trên đồ thị có trọng số.

Mục tiêu bài học

- Duyệt đồ thị theo chiều rộng (BFS) và chiều sâu (DFS)
- Dùng BFS tìm đường đi ít cạnh nhất trên đồ thị không trọng số
- Tìm đường đi ngắn nhất trên đồ thị có trọng số không âm bằng Dijkstra

1. Duyệt theo chiều rộng (BFS)

BFS bắt đầu từ một đỉnh, thăm hết các đỉnh kề trước rồi mới đi xa hơn, dùng hàng đợi (queue). Các đỉnh được thăm theo thứ tự khoảng cách (số cạnh) tăng dần, nên BFS tìm được đường đi ít cạnh nhất trên đồ thị không trọng số.

```
from collections import deque

def bfs(graph, start):
    visited, queue = {start}, deque([start])
    while queue:
        u = queue.popleft()
        for v in graph[u]:
            if v not in visited:
                visited.add(v)
                queue.append(v)
    return visited
```

2. Duyệt theo chiều sâu (DFS)

DFS đi sâu theo một nhánh đến khi không đi tiếp được rồi quay lui, dùng ngăn xếp (stack) hoặc đệ quy. DFS dùng để kiểm tra liên thông, phát hiện chu trình, sắp xếp tô pô. Cả BFS và DFS có độ phức tạp $O(|V| + |E|)$ với danh sách kề.

3. Thuật toán Dijkstra

Dijkstra tìm đường đi ngắn nhất từ một đỉnh nguồn đến mọi đỉnh trên đồ thị có trọng số KHÔNG ÂM. Ý tưởng: luôn chọn đỉnh chưa xét có khoảng cách tạm thời nhỏ nhất, chốt khoảng cách của nó, rồi cập nhật (nới lỏng) các đỉnh kề: nếu $d[u] + w(u, v) < d[v]$ thì gán $d[v] = d[u] + w(u, v)$.

Dùng hàng đợi ưu tiên (heap), độ phức tạp khoảng $O((|V| + |E|) \log |V|)$. Khi đồ thị có cạnh trọng số âm, Dijkstra có thể cho kết quả sai vì một đỉnh đã chốt vẫn có thể được rút ngắn sau đó; trường hợp này dùng thuật toán Bellman–Ford.

Tóm tắt

- BFS dùng hàng đợi, tìm đường ít cạnh nhất; DFS dùng ngăn xếp / đệ quy.
- Dijkstra: chọn đỉnh có khoảng cách nhỏ nhất, nối lỏng các cạnh kề.
- Dijkstra chỉ đúng với trọng số không âm; có cạnh âm dùng Bellman–Ford.

Bài tập vận dụng

1. Ghi thứ tự thăm đỉnh khi chạy BFS và DFS từ đỉnh A trên đồ thị bài trước.
2. Chạy tay Dijkstra trên đồ thị 4 đỉnh: A–B (1), A–C (4), B–C (2), C–D (1). Tìm khoảng cách từ A đến D.